This interface provides a lot of flexibility, but we don't need much of it in our Invoice class. It would be nice to have a simpler API that fits just the needs of our project.

Look what happens when we try to test this class. We end up creating one test double, tax_client, that returns a second double, tax_rate:

15-using-test-doubles-effectively/22/sales_tax/spec/unit/invoice_spec.rb

```
require 'invoice'

RSpec.describe Invoice do
  let(:address) { Address.new(zip: '90210') }
  let(:items) { [Item.new(cost: 30), Item.new(cost: 70)] }

  it 'calculates the total' do
➤   tax_rate = instance_double(Taxjar::Rate, combined_rate: 0.095)
➤   tax_client = instance_double(Taxjar::Client, rates_for_location: tax_rate)

    invoice = Invoice.new(address, items, tax_client: tax_client)

    expect(invoice.calculate_total).to eq(109.50)
  end
end
```

Our complex mocking structure (a double that returns a double) has revealed a couple of problems with our project:

- The Invoice class is tightly coupled to the TaxJar client's object hierarchy.
- The complexity will make refactoring and maintenance more difficult.

In his blog post, "Test Isolation Is About Avoiding Mocks," Gary Bernhardt recommends that we untangle the object relationships that are made obvious by these kinds of nested doubles.[15] We can follow his advice by writing a SalesTax wrapper that decouples Invoice from TaxJar:

15-using-test-doubles-effectively/23/sales_tax/lib/sales_tax.rb

```
require 'my_app'

class SalesTax
  RateUnavailableError = Class.new(StandardError)

  def initialize(tax_client = MyApp.tax_client)
    @tax_client = tax_client
  end

  def rate_for(zip)
    @tax_client.rates_for_location(zip).combined_rate
  rescue Taxjar::Error::NotFound
    raise RateUnavailableError, "Sales tax rate unavailable for zip: #{zip}"
  end
end
```

15. https://www.destroyallsoftware.com/blog/2014/test-isolation-is-about-avoiding-mocks